

Causal & Padding Masks, Worked Out by Hand

Adding minus-infinity before softmax to forbid the future and ignore padding

A complete numerical walk-through of attention masking. We take one small 3×3 matrix of attention scores, add an upper-triangular block of $-\infty$ to forbid each position from looking at the future, and watch the row-softmax collapse those entries to *exactly* zero — producing a lower-triangular weight matrix that *is* autoregression. Then we mask a padded key position the same way. Every number below was produced by the NumPy/PyTorch reference program in Appendix A, so the arithmetic is exact and self-consistent.

What this document does. It implements the course's two masking recipes by hand. The causal mask, `torch.triu(ones, diagonal=1)`, blocks position i from attending to any $j > i$; the padding mask, `(tokens == pad_id)`, blocks key positions that are only there to square off a ragged batch. Both reduce to a single operation — `scores.masked_fill(mask, -inf)` before `softmax` — and we trace that operation entry by entry.

How to read this. Each mask gets its own worked section with the raw scores, the masked scores, the per-row exp arithmetic, and the final weights. The *Mini-example* boxes isolate a single row so you can see $e^{-\infty} = 0$ doing all the work. This builds directly on the previous topic, Scaled Dot-Product Attention: masking happens *between* the scaled scores and the softmax, changing nothing else.

Concepts covered, in order: the scores→mask→softmax pipeline · why $-\infty$ and not a large negative number · the causal (look-ahead) mask and its lower-triangular weights · how masked weights become exactly 0 via $e^{-\infty} = 0$ · the output WV at the first position equals its own value · the padding (key) mask for ragged batches · combining causal + padding · the train/serve leak you get if you forget the causal mask.

1 The setup: where masking lives in attention

Recall the scaled dot-product attention pipeline. From queries Q and keys K we form a $T \times T$ matrix of *scores*, scale it, and pass each row through a softmax to get a probability distribution over which positions to read:

$$S = \frac{QK^\top}{\sqrt{d_k}}, \quad W = \text{softmax}_{\text{row}}(S), \quad O = WV.$$

A **mask** slots in between S and the softmax. It does exactly one thing: for every position we are not allowed to attend to, it overwrites that score with $-\infty$. The course writes this as a single line,

$$S \leftarrow S.\text{masked_fill}(\text{mask}, -\infty), \quad W = \text{softmax}_{\text{row}}(S)$$

Everything else — the scaling, the V multiply — is untouched.

We work the entire document on one fixed 3×3 score matrix (already scaled, so we can read the numbers cleanly):

$$S = \begin{bmatrix} 2.0 & 1.0 & 0.0 \\ 0.5 & 1.5 & 1.0 \\ 1.0 & 0.0 & 2.0 \end{bmatrix}.$$

Row i holds the scores that *query* i assigns to each of the three *key* positions 0, 1, 2. With **no mask**, every row softmaxes over all three keys:

$$W_{\text{full}} = \begin{bmatrix} 0.665 & 0.245 & 0.090 \\ 0.186 & 0.506 & 0.307 \\ 0.245 & 0.090 & 0.665 \end{bmatrix}.$$

Each row sums to 1 and every position can see every other position — including the future. For an encoder classifying a complete, already-known sequence that is fine. For a *decoder* that must generate left-to-right, it is a bug.

Intuition

A mask is not a special mechanism — it is just adding $-\infty$ to the scores you want forbidden. Softmax exponentiates, and $e^{-\infty} = 0$, so those entries get exactly zero weight and drop out of the weighted sum. The surviving entries renormalise among themselves. Everything downstream (the WV multiply, the gradients) runs unchanged; you have simply deleted some edges from the attention graph before the probabilities are formed.

2 Step 1 — The causal mask: forbidding the future

In autoregressive decoding, token i is predicted from tokens $0, \dots, i$ only. So query i may attend to keys $0, \dots, i$ but *not* to any $j > i$. The forbidden set is precisely the strict upper triangle, which is what `torch.triu(torch.ones(T,T), diagonal=1)` marks. Setting those scores to $-\infty$:

$$S \xrightarrow{\text{mask } j>i} S_c = \begin{bmatrix} 2.0 & -\infty & -\infty \\ 0.5 & 1.5 & -\infty \\ 1.0 & 0.0 & 2.0 \end{bmatrix}.$$

Now apply the row-softmax. Softmax of a row r is computed stably as $w_k = e^{r_k - m} / \sum_{\ell} e^{r_{\ell} - m}$ with $m = \max_{\ell} r_{\ell}$, and crucially the $-\infty$ entries contribute $e^{-\infty} = 0$ to both the numerator and the sum.

Row 0 (only key 0 survives):

$$m = 2.0, \quad [e^0, e^{-\infty}, e^{-\infty}] = [1, 0, 0], \quad \text{sum} = 1, \quad w_0 = [1.0, 0.0, 0.0].$$

Row 1 (keys 0, 1 survive; key 2 forbidden):

$$m = 1.5, \quad [e^{0.5-1.5}, e^{1.5-1.5}, 0] = [0.368, 1.0, 0], \quad \text{sum} = 1.368,$$

$$w_1 = \left[\frac{0.368}{1.368}, \frac{1.0}{1.368}, 0 \right] = [0.269, 0.731, 0.0].$$

Row 2 (all three keys allowed — nothing is in its future):

$$m = 2.0, \quad [e^{-1.0}, e^{-2.0}, e^0] = [0.368, 0.135, 1.0], \quad \text{sum} = 1.503,$$

$$w_2 = [0.245, 0.090, 0.665].$$

Stacking the rows gives a **lower-triangular** weight matrix:

$$W_c = \begin{bmatrix} 1.000 & 0.000 & 0.000 \\ 0.269 & 0.731 & 0.000 \\ 0.245 & 0.090 & 0.665 \end{bmatrix} \quad \text{every row still sums to 1.}$$

The strict upper triangle is exactly zero, not merely small: $e^{-\infty}$ is a true zero, so no probability mass leaks to the future. Notice row 2 of W_c equals row 2 of W_{full} — the last position has no future to hide, so its mask is empty.

Mini-example — this operation in isolation

Watch a single masked entry vanish. Row 1 before masking is $[0.5, 1.5, 1.0]$, whose full softmax is $[0.186, 0.506, 0.307]$ — key 2 would get 30.7% of the weight. Masking key 2 replaces its score with $-\infty$, so its exp term is 0 and it is removed *before* normalisation. The remaining two terms $[0.368, 1.0]$ renormalise over their own sum 1.368, giving $[0.269, 0.731, 0]$. Masking is not "set the weight to zero afterward"; it is "delete the term before the sum," which is why the survivors still add to exactly 1.

Watch out

Use $-\infty$ (`float('-inf')`), not a merely large negative like -10^9 . With a finite stand-in the masked weight is tiny but *nonzero*, and across long sequences thousands of those leaks accumulate. PyTorch's `masked_fill(mask, float('-inf'))` and `softmax` are built to handle true $-\infty$ cleanly: $e^{-\infty} = 0$ exactly, and the max-subtraction keeps the finite entries stable.

3 Step 2 — What the masked weights do: the WV output

The weights are only half the story; they multiply the value vectors. Take three value vectors (one per position), $d_v = 2$:

$$V = \begin{bmatrix} 1 & 0 \\ 0 & 2 \\ 4 & 1 \end{bmatrix}.$$

The causal output is $O = W_c V$, one row per query:

$$O = \begin{bmatrix} 1.000 & 0.000 \\ 0.269 & 1.462 \\ 2.906 & 0.845 \end{bmatrix}.$$

Read row 0: query 0 has weights $[1, 0, 0]$, so $O_0 = 1 \cdot V_0 = [1, 0]$ — the very first token can only attend to itself. Row 1 mixes V_0, V_1 by $0.269/0.731$: $0.269[1, 0] + 0.731[0, 2] = [0.269, 1.462]$. Row 2 mixes all three. The mask has turned a fully-connected attention into a strictly causal one, position by position.

Intuition

The lower-triangular weight matrix *is* autoregression written as one matrix multiply. During training we feed the whole sequence at once and compute all T rows in parallel — yet because row i is zero past column i , row i depends only on tokens $0, \dots, i$, exactly as if we had generated them one at a time. Parallel training and one-token-at-a-time generation compute the *same* function. That equivalence is the whole reason Transformers train fast and still generate correctly.

When this is the right choice

Reach for the **causal** mask whenever the model must generate or predict the next token left-to-right: GPT-style decoders, language-model heads, the decoder side of seq2seq, any "next-event prediction." Reach for the **padding** mask (next section) whenever you batch sequences of different lengths — which, in practice, is almost always. Decoders that also batch ragged inputs need *both* at once (Step 4).

4 Step 3 — The padding mask: ignoring filler tokens

To batch sequences of different lengths into one tensor, short ones are padded to the common length with a pad id (the course uses 0, with `padding_mask = (tokens == 0)`). Those pad slots carry no information, so no query should ever put weight on them. The padding mask blocks the padded *key* columns.

Take one query attending over a sequence padded to length 4, where key 3 is padding. Raw scores over the four keys:

$$s = [1.0, 2.0, 0.5, 1.5].$$

Without masking, the softmax wastes weight on the pad slot:

$$\text{softmax}(s) = [0.167, 0.455, 0.102, 0.276] \Rightarrow 27.6\% \text{ of attention spent on padding.}$$

Masking the pad key sets $s_3 = -\infty$:

$$\begin{aligned} s' &= [1.0, 2.0, 0.5, -\infty], & m &= 2.0, \\ [e^{-1.0}, e^0, e^{-1.5}, e^{-\infty}] &= [0.368, 1.0, 0.223, 0.0], & \text{sum} &= 1.591, \\ \text{softmax}(s') &= [0.231, 0.629, 0.140, 0.000] \end{aligned}$$

The pad slot now receives *exactly* 0.0, and the three real keys renormalise over their own mass to sum to 1. The padded token has been made invisible.

Mini-example — this operation in isolation

Same mechanism, different geometry. The causal mask removes entries by *position relative to the query* (a triangular pattern, different cells per row). The padding mask removes entries by *which key column is filler* (a vertical stripe, the same columns for every row). Both are just "score $\rightarrow -\infty$ before softmax"; only the *set* of cells differs.

5 Step 4 — Combining causal and padding masks

A batched decoder needs both: forbid the future *and* ignore padding. You simply block a cell if *either* rule forbids it. With $T = 4$, positions 0, 1, 2 real and position 3 padding, the "allowed" pattern (1 = keep) is the causal lower triangle with column 3 knocked out:

$$\text{allow} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 0 \end{bmatrix} \Rightarrow W = \begin{bmatrix} 1.000 & 0.000 & 0.000 & 0.000 \\ 0.269 & 0.731 & 0.000 & 0.000 \\ 0.245 & 0.090 & 0.665 & 0.000 \\ 0.333 & 0.333 & 0.333 & 0.000 \end{bmatrix}.$$

Rows 0–2 reproduce the causal weights of Step 1 (the pad column was already in their future or just zeroed), and column 3 is zero everywhere because the pad key is masked for all queries. Row 3 is the *pad query* itself: its output is discarded by the loss, so any normalised weights there are harmless (here the surviving keys 0, 1, 2 split evenly because their post-mask scores are all equal).

Watch out

Omitting the causal mask in a decoder **leaks the future**: at training time each position quietly peeks at the answer (tokens to its right), loss looks great, and then at generation time — where the future genuinely does not exist — the model collapses. This train/serve mismatch is silent and brutal. Rule of thumb: a generative decoder needs the causal mask *always*, and the padding mask *whenever* the batch is ragged, so you usually combine both.

6 The two masks, at a glance

	Causal mask	Padding mask
Forbids	future keys $j > i$	padded key columns
Pattern	strict upper triangle	vertical stripe(s)
Built by	<code>triu(ones, diagonal=1)</code>	<code>(tokens == pad_id)</code>
Shape	(T, T) , same per batch item	(B, T) , per-row
Use when	decoding / generation	batching ragged lengths
Weights become	lower-triangular	zero on pad columns
Forget it and...	future leaks (train/serve gap)	attention wasted on filler

Both are the identical operation underneath — `masked_fill(..., -inf)` then `softmax` — applied to a different set of cells. Print the post-softmax weight matrix when debugging: a correct causal mask shows a clean lower triangle, and a correct padding mask shows dead columns.

7 Equation cheat-sheet

Scaled scores	$S = QK^\top / \sqrt{d_k}$
Apply mask	$S_{ij} \leftarrow -\infty$ for every forbidden (i, j)
Causal rule	forbid $j > i$ ($\Leftrightarrow \text{triu}(\text{ones}, \text{diagonal}=1)$)
Padding rule	forbid key column j if token j is pad
Row-softmax	$w_{ij} = \frac{e^{S_{ij}-m_i}}{\sum_\ell e^{S_{i\ell}-m_i}}, \quad m_i = \max_\ell S_{i\ell}$
Masked entry	$e^{-\infty} = 0 \Rightarrow w_{ij} = 0$ exactly
Output	$O = WV$
Causal weights	lower-triangular; each row sums to 1

Appendix A Reference implementation

Every number above is reproducible from the following program. The NumPy `masked_softmax` mirrors the math line for line; the PyTorch block uses the exact course idioms (`triu(...,diagonal=1)` and `masked_fill(mask, -inf)`).

```
import numpy as np

def masked_softmax(scores):
    # scores may contain -inf for masked cells; softmax over the last axis
    m = np.max(np.where(np.isfinite(scores), scores, -np.inf), axis=-1, keepdims=True)
    e = np.where(np.isfinite(scores), np.exp(scores - m), 0.0)
    return e / e.sum(axis=-1, keepdims=True)

# --- (A) CAUSAL MASK on a fixed 3x3 score matrix ---
S = np.array([[2.0, 1.0, 0.0],
              [0.5, 1.5, 1.0],
              [1.0, 0.0, 2.0]])
print(masked_softmax(S).round(3))          # full attention (no mask)

Sc = S.copy()
for i in range(3):
    for j in range(3):
        if j > i:
            # strict upper triangle = the future
            Sc[i, j] = -np.inf
Wc = masked_softmax(Sc)
print(Wc.round(3))          # lower-triangular causal weights
# [[1.    0.    0. ]
#  [0.269 0.731 0. ]
#  [0.245 0.09  0.665]]

V = np.array([[1.0, 0.0], [0.0, 2.0], [4.0, 1.0]])
print((Wc @ V).round(3))    # output O = Wc @ V
# [[1.    0.   ] [0.269 1.462] [2.906 0.845]]

# --- (B) PADDING MASK: one query over 4 keys, key 3 is PAD ---
s = np.array([1.0, 2.0, 0.5, 1.5])
print(masked_softmax(s).round(3))          # [0.167 0.455 0.102 0.276]
s_masked = s.copy(); s_masked[3] = -np.inf
print(masked_softmax(s_masked).round(3))   # [0.231 0.629 0.14  0.   ]

# --- the same thing in PyTorch, course idioms ---
import torch
S = torch.tensor([[2.,1.,0.],[0.5,1.5,1.],[1.,0.,2.]])
causal = torch.triu(torch.ones(3, 3), diagonal=1).bool() # True = block
Sc = S.masked_fill(causal, float('-inf'))
print(torch.softmax(Sc, dim=-1).round(decimals=3))

# padding: tokens == pad_id marks pad columns
tokens = torch.tensor([5, 9, 7, 0])          # last token is pad (id 0)
pad = (tokens == 0)                          # True where padding
s = torch.tensor([1., 2., 0.5, 1.5])
print(torch.softmax(s.masked_fill(pad, float('-inf')), dim=-1).round(decimals=3))
# tensor([0.2310, 0.6290, 0.1400, 0.0000])
```